

Mealody — Project Guide & Interview Notes

2026-09-17 · @Someone

What Mealody is

Mealody is a web app that plans a full week of meals around what you already have in your kitchen.

The problem it solves. Most people know the 6pm question: "what's for dinner?" Recipe sites make you search dish by dish, and meal-planning apps hand you a generic plan built around ingredients you'd have to go out and buy. Neither one starts from what's actually sitting in your fridge.

What Mealody does. You tell it what you have, how you're feeling, and what you avoid. It generates a week of breakfast, lunch and dinner built around those ingredients — 21 meals, each with a short reason it was chosen and a full recipe behind it. You can save the ones you love, add your own family recipes, and send ingredients straight to a shopping list organized by store.

Who it's for. Home cooks who want less decision fatigue — especially people cooking for a household, using up leftovers, or working around dietary needs and allergies.

Where it lives. It runs at nusrathfathima.com as part of my portfolio.

How someone uses it

1. A short conversation. Instead of a long form, Mealody asks four questions one at a time, chat style: what's in your kitchen, any leftovers, what vibe you're after, and which cuisines you fancy. Each answer gets a friendly reply so it feels like talking to someone rather than filling in a survey. Diet, servings and allergies live in Settings, since those rarely change week to week.

2. The week appears. Days arrive progressively rather than all at once, so there's something to look at within seconds. Each meal card shows the dish, a one-line reason it was picked, and its key ingredients.

3. Tweaks. The ↻ button on any card suggests a different dish for that slot. A heart saves a dish to Favorites. If a day fails to generate, it can be retried on its own without redoing the week.

4. Recipes. Tapping a card opens the full recipe — timing, ingredients scaled to the number of people, numbered steps and a chef's tip.

5. Saving and reusing. Favorites holds dishes you hearted, plus "My recipes" where you add your own family dishes (typed yourself, or written by AI). Anything you mark with + gets woven into future weeks — but only when its ingredients actually fit what you've picked.

6. Shopping. Ingredients go to a grocery list with one tap, organized by store. Items can be checked off, given quantities and notes, swiped away on mobile, and the whole list shared to WhatsApp or anywhere else.

7. Exports. The week can be saved as a PDF (with or without full recipes) or as a shareable image.

How it's built

The short version: a single HTML file of vanilla JavaScript, one small serverless function, and Claude Haiku doing the thinking. No framework, no database, no build step.

The pieces

Piece	What it does
<code>index.html</code>	The entire app — markup, CSS and JavaScript in one file
<code>api/generate.js</code>	A serverless function on Vercel that talks to the AI
Claude Haiku	Generates the meal plans and recipes
Browser localStorage	Stores favorites, recipes, grocery lists and settings
Vercel	Hosting and automatic deploys from GitHub

How a request flows

The browser sends the user's answers to my own `/api/generate` endpoint. That function adds the secret API key and forwards the request to Anthropic, then passes the answer back. The key never reaches the browser, which is the whole reason the function exists — calling the AI directly from the page would expose it to anyone who opened DevTools.

Why the week is generated in pieces

Asking for 21 meals in one request was slow enough to risk hitting the serverless timeout, and a failure meant losing everything. So the week is built in stages: first the theme, then days in small batches. Each batch renders the moment it arrives, so the page fills up progressively instead of showing a spinner for 30 seconds. If one batch fails, only those days are affected and they can be retried on their own.

Recipes

Recipes aren't generated with the plan, because most people never read all 21. The first time someone taps a card, that recipe is fetched immediately and a background queue quietly fills in the rest, two days per request. Everything gets cached in the browser by dish name, so a dish you've seen before costs nothing the next time.

Storage

Everything the user saves lives in the browser's `localStorage` under a handful of keys — favorites, their own recipes, grocery lists, settings, the cached recipes, and the current week. There's no login and no database, which keeps the app free to run and instant to use.

Pages without a router

Favorites and Groceries are full pages reached through the URL hash (`#/groceries/...`). The browser back button works, links can be shared, and refreshing lands you in the right place — all without pulling in a routing library.

Talking points for interviews

These are the decisions worth raising. For each one, the pattern is the same: *here's what I chose, here's why, here's what it cost me*. Interviewers care far more about the reasoning than the feature.

Protecting the API key

The obvious way to call an AI from a web page is to call it directly — and it's wrong, because the key ends up in the browser where anyone can take it and spend your money. I put a small serverless function in between: the browser calls my endpoint, the endpoint

adds the key server-side. I also clamped the maximum response size on the server, so even someone hitting my endpoint directly can't demand huge expensive responses.

Talking point: "I treated my own frontend as untrusted."

Splitting the generation

Generating 21 meals in one call was slow and fragile — one timeout lost the whole week. Splitting it into batches made it faster to first content, gave partial results instead of total failure, and let a single failed day be retried on its own.

Talking point: "I traded one big request for several small ones, and got resilience and perceived speed out of it."

Only paying for what people use

Output tokens cost about five times what input tokens do, and recipes are almost entirely output. So rather than generating all 21 recipes up front, nothing is generated until someone first taps a card — then the rest fill in quietly in the background. Recipes are cached by dish name, so repeat dishes are free. Prompts explicitly ask for tight output: eight steps maximum, one sentence each, no filler.

Talking point: "I cut the cost of the feature roughly in half without removing anything users see."

Choosing boring technology

No React, no build step, no database. For an app this size that meant no toolchain to maintain, instant deploys, and a page that loads fast. The honest tradeoff: a single large HTML file gets harder to navigate as it grows, and I'd reach for components if it got much bigger.

Talking point: "I picked the smallest thing that worked, and I can tell you exactly when I'd outgrow it."

localStorage, knowingly

Saved dishes and grocery lists live in the browser. That means no login, no server cost, instant reads — but data stays on one device and vanishes if browsing data is cleared. I chose it deliberately for a portfolio app, and I know exactly what replacing it looks like (see the next section).

Talking point: "I can name the limitation before you do, and I've already planned the migration."

Designing for touch

Phones don't have hover, but they *latch* the hover state after a tap — which is why a button on my site stayed visibly tilted after being pressed. I now wrap hover effects so touch devices get press feedback instead. Grocery items can be swiped away, with the swipe locked to the horizontal axis so it doesn't fight scrolling.

Talking point: "Mobile isn't just narrower — the interaction model is different."

Checking contrast rather than trusting my eyes

Building dark mode, I measured contrast ratios instead of eyeballing them. Two elements were effectively invisible: light text on a light panel at a ratio of 1.03, and white text on a light red button at 2.52 (4.5 is the readable threshold). Both looked fine at a glance on my monitor.

Talking point: "Accessibility is measurable, so I measured it."

Problems I hit, and how I fixed them

Interviewers love these. Each one is a small story with a cause and a fix — much more convincing than a list of features.

The AI kept suggesting meat nobody asked for

Symptom: a user picks chicken, and the week comes back with pork and lamb dishes.

Cause: my own prompt. It said "vary proteins day to day," which the model read as permission to introduce new ones.

Fix: removed that phrase and replaced it with a hard rule — only meats the user listed may appear, and variety must come from rotating those with vegetarian meals instead.

Lesson: when an AI misbehaves, suspect the instructions before the model.

Saved dishes appearing in the wrong week

Symptom: a saved paneer dish kept showing up in a week built around chicken, even though the prompt said not to.

Cause: I was asking the model to filter. Models don't reliably follow negative instructions like "leave it out if it doesn't fit."

Fix: moved the decision into code. Before anything is sent, each saved dish is checked against the kitchen, the diet and the allergies — a dish whose signature ingredient isn't available never reaches the model at all.

Lesson: if a rule must always hold, enforce it in code. Use the AI for judgement, not for guarantees.

"Non-vegetarian" contains "vegetarian"

Symptom: while testing that filter, meat dishes were being stripped out of non-veg weeks.

Cause: my check was `/vegetarian/.test(diet)` — which matches inside the word "Non-vegetarian."

Fix: test for "non-veg" first, then fall through to vegetarian.

Lesson: a substring match on user-facing labels is a trap. This one only surfaced because I wrote test cases for the filter rather than clicking through the UI.

An invisible button in dark mode

Symptom: the + button on saved dish cards vanished in dark mode.

Cause: the CSS set a background but no text colour. Buttons don't inherit colour from their parent — they fall back to the browser's near-black default, which disappeared against a dark card.

Fix: an explicit colour, plus a light translucent surface in dark mode. Auditing for the same mistake found two more places.

Lesson: one bug report is often a symptom of a pattern. Go looking for its siblings.

A menu you could see straight through

Symptom: the mobile nav menu opened with the page's headline showing through it.

Cause: not transparency — stacking order. My layout class set `z-index: 1` on every section, so the hero and the header were siblings at the same level, and the hero (later in the document) painted over the header entirely. The menu's own high z-index was trapped inside the header's context and never got compared.

Fix: raised the header above the page content.

Lesson: z-index is relative to the stacking context, not the page.

Two components fighting over one class name

Symptom: the diet selector's three buttons collapsed into a squashed layout.

Cause: I'd named the new Favorites tabs `.seg` — the same class the diet control already used. My two-column grid silently overrode its three-column one.

Fix: renamed mine, and started prefixing new component classes.

Lesson: in a single-file app, the global namespace is real and it will bite.

What's next

A backend for saved recipes and groceries (the planned next step)

Why. Right now everything is saved in the browser. That means your favorites and shopping lists live on one device — build a list on your laptop and it isn't on your phone at the shop — and clearing browsing data wipes them. A backend fixes both.

What it needs, roughly in order:

1. **Accounts.** Sign-in, ideally through a hosted service (Supabase Auth, Clerk, Auth0) rather than rolling password handling myself.
2. **A database.** Tables for users, saved dishes, personal recipes, grocery lists and list items. Postgres through Supabase is a natural fit, and its free tier suits a portfolio project.
3. **API endpoints.** More serverless functions alongside the existing one, to read and write that data, with every request checked so one user can't touch another's rows.
4. **Migration.** On first sign-in, offer to upload whatever's already in the browser so existing users don't lose anything.
5. **A sync strategy.** Keep writing to `localStorage` first so the app stays instant, then sync to the server in the background — rather than making every tap wait on the network.

The interesting hard part, and a good thing to be able to discuss: what happens when the same list is edited on two devices. The simplest answer is last-write-wins per item; a better one tracks changes per row. Knowing that this question exists is worth more in an interview than having solved it.

What it unlocks beyond sync: sharing a grocery list with a partner so both phones see the ticks, plan history across weeks, and a real path to user accounts if it ever became a product.

Other ideas, roughly by value

A pantry that remembers. Today you retype your ingredients every week. A saved pantry would prefill that first question, and checked-off grocery items could flow back into it automatically — closing the loop between all three parts of the app.

Cook mode. Open a recipe, tap Cook, and get a full-screen step-by-step view with big text, the screen kept awake, and timers where the recipe mentions minutes. This is what makes the app useful *at the stove* rather than only at the planning stage.

Installable app (PWA). A manifest and a service worker would let people add Mealody to their home screen and open grocery lists offline — exactly when signal is worst, standing in a shop. Small effort, and a concrete line on a CV.

Smaller things: swapping a whole day rather than one meal; rough nutrition estimates; leftover-aware planning ("Tuesday's dal becomes Wednesday's filling"); a printable grocery list to match the recipe PDFs.

Honest limitations today

- Data is per-device and per-browser.
- Generated plans depend on a live connection; nothing can be created offline.
- The AI is probabilistic — most rules are enforced in code now, but output still varies run to run.
- Everything lives in one large HTML file, which is fine at this size and wouldn't be at ten times it.

Quick answers to likely questions

Short versions to have ready. Say them in your own words — the point is having the answer, not reciting it.

"Walk me through this project." Mealody plans a week of meals around what's already in your kitchen. You answer four quick questions, it generates 21 meals with recipes, you save the ones you like and send the ingredients to a shopping list. It's vanilla JavaScript on the front end with a small serverless function that calls Claude, hosted on Vercel.

"Why no React?" For an app this size it would have added a build step and a toolchain without making anything better. I wanted fast loads and simple deploys. I'd reach for components once the shared state got complex enough that passing it around by hand became error-prone — which is roughly where this app is heading if it grows.

"How do you keep your API key safe?" It never touches the browser. The page calls my own endpoint, and that function adds the key server-side before calling Anthropic. I also cap the maximum response size on the server so the endpoint can't be abused to run up a bill.

"How do you handle AI costs?" Four things: recipes are only generated when someone actually opens one, they're cached by dish so repeats are free, requests are batched two days at a time, and prompts explicitly ask for short output because output tokens cost several times more than input. Together those roughly halved the cost of the recipe feature.

"What happens when the AI fails?" It's built in from the start. The week generates in batches, so one failure affects a few days rather than everything, and those days can be retried on their own. Failed responses show a friendly message with a retry rather than an error. And anything that must always be true — diet rules, allergies, matching ingredients — is enforced in code, not left to the model.

"What was the hardest bug?" Saved dishes appearing in weeks they didn't belong in. I'd written a prompt telling the model to leave them out when they didn't fit, and it didn't reliably obey. The fix was to stop asking: the filtering now happens in code before the request is sent. Writing test cases for that filter then caught a second bug — my check for "vegetarian" was also matching inside "non-vegetarian."

"What would you do differently?" Split the code into modules earlier — one file was right at the start and is now large. I'd also have written small tests for the pure logic sooner; the ones I did write found real bugs immediately.

"What's the biggest limitation?" Data lives in the browser, so it doesn't sync between devices. That was a deliberate tradeoff to keep the app free and login-free, and adding a backend is the next planned step — accounts, a Postgres database, and background sync so the app still feels instant.

"How did you test it?" Manually across browsers and devices, plus automated checks on the pure logic — ingredient parsing, the dish-matching filter, input validation — and measured colour-contrast ratios for dark mode rather than judging by eye.